

软件工程设计模式报告

旅馆客房管理系统 设计模式报告

陈佳伟 孙耀珠 余阅来 陈钰 杨宇轩 王禹诚
2018.5.27

目录

I. 最新文献回顾.....	3
II. 系统体系分析.....	3
III. 经典 GOF 设计模式应用	3
1. 创建型模式	4
a. 抽象工厂 (<i>Abstract Factory</i>)	4
b. 工厂方法 (<i>Factory Method</i>)	5
c. 原型 (<i>Prototype</i>)	6
d. 单件 (<i>Singleton</i>)	7
2. 结构型模式	8
a. 适配器 (<i>Adaptor</i>)	8
b. 桥接 (<i>Bridge</i>)	9
c. 外观 (<i>Facade</i>)	10
d. 代理 (<i>Proxy</i>)	11
3. 行为模式	12
a. 解释器 (<i>Interpreter</i>)	12
b. 迭代器 (<i>Iterator</i>)	14
c. 中介者 (<i>Mediator</i>)	15
d. 观察者 (<i>Observer</i>)	16
e. 状态 (<i>State</i>)	16
f. 策略 (<i>Strategy</i>)	17
g. 模板方法 (<i>Template Method</i>)	18
IV. 参考文献	19

I. 最新文献回顾

软件工程中的设计模式研究最早可追溯到 GoF 所著的《设计模式》，而如今设计模式的研究方向相对于早期的则有所调整。

近几年，软件工程中设计模式的自动识别成为一些研究者所关注的领域。有的研究者通过把源代码转化为图的方式进行识别，但时间复杂度较高[1]；另有研究者采用实时识别的方法进行设计模式识别[1]。

同时，有关本体论设计模式（Ontology Design Patterns）的研究[1][5]也逐渐兴起，研究者利用“本体”为软件应用程序提供精确的领域模型[4]。

II. 系统体系分析

本项目是围绕着数据的增删改查的数据中心模型，项目的主要操作是有关数据的操作。因此我们的设计模式也应该围绕着这一特点展开。同时，由于我们的项目围绕着宾馆展开，宾馆中的房间、物品、账单、人员等都可以用面向对象的思想进行建模，从而我们可以应用的设计模式也会涉及一些 OO 的内容。由于我们使用 Java 进行相关代码编写，这在不知不觉中其实已经用到许多设计模式。

III. 经典 GoF 设计模式应用

设计模式所提倡的一个原则是“相比于继承，我们更优先使用组合”。正是在这一原则的指导下，我们可以采用一些典型的设计模式去替代我们原有的一些设计理念。许多设计模式都遵循着这一法则。

本部分我们主要沿着 GoF 所著的设计模式书籍《设计模式：可复用面向对象软件的基础》[1]来展开我们的设计模式应用研究。

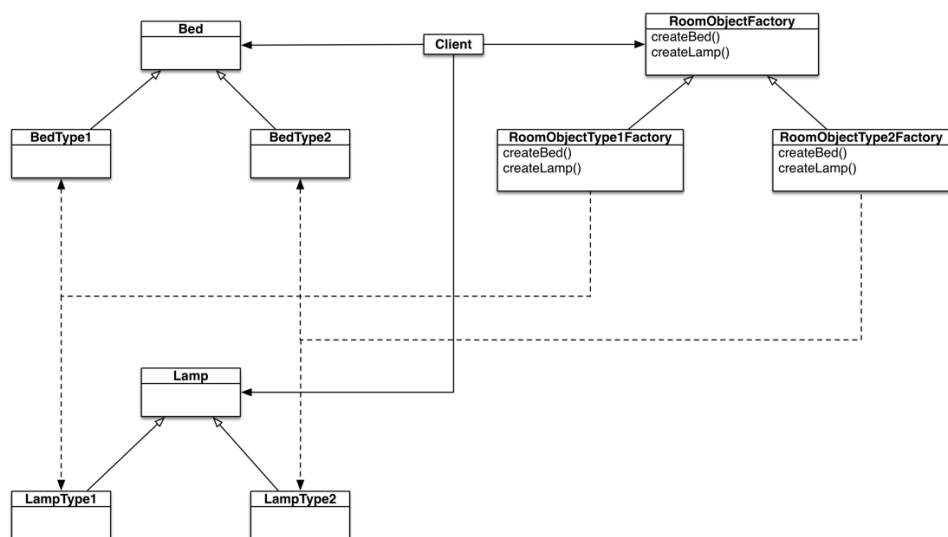
1. 创建型模式

a. 抽象工厂（Abstract Factory）

抽象工厂的意图是“提供一个创建一系列相关或相互依赖对象的接口，而无需指定它们具体的类”。

抽象工厂的优点是可扩展，如本例中，如果要添加新的房间物品类型，只需要增加一个抽象工厂类的子类即可。

我们旅馆中的房间中有各种物件，房间类可以看作是其他的物件类组合而成的。在这里我们可以采用抽象工厂方法，设置抽象工厂类，来进行有关类的实现。对于 bed 和 lamp 的各种类型，client 通过调用 RoomObjectFactory 进行生产。这样的配置带来一个好处，当我们需要增加一种新的风格时，我们只需要为 RoomObjectFactory 增加一个子类，使用这个子类来进行新风格的 bed 和 lamp 的生产。这样以来房间中的各个物品不是硬编码的，而是解耦的。

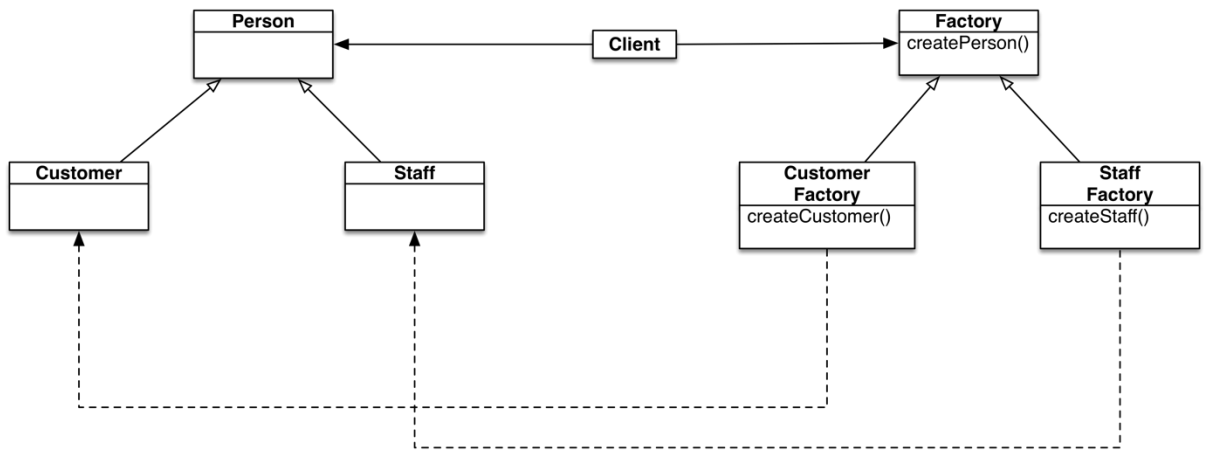


b. 工厂方法（Factory Method）

工厂方法的意图是将类的实例化延迟至子类，让子类决定实例化哪一个类。工厂方法使得创建对象的操作更加灵活而不必局限于类的直接创建。

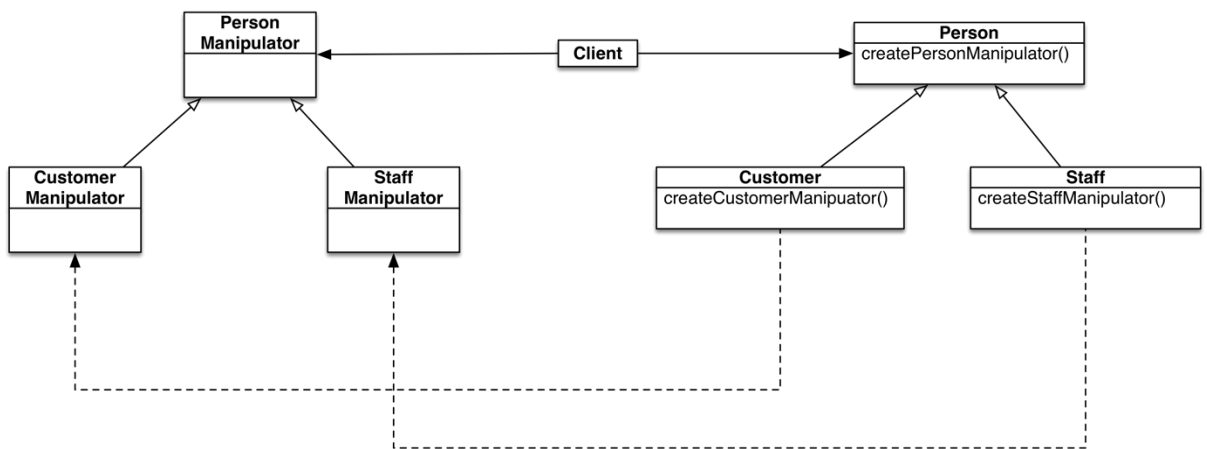
构造函数是类的实例化的基本方法，但是构造函数有其固有缺陷，例如构造函数的名称只能与类名一致。但是如果采用工厂方法则可以自己定义新的名称，比较灵活。

我们的房间、人员等信息存放在数据库中，我们可以单独设立一个类来实现这些类的实例化，而不是通过直接调用这些类自己的构造函数来实例化。对于人员类来说，其子类的实例化通过调用工厂类的子类来进行实例化。在这里，Person 类及其子类的构造函数被忽略，代替它们的是工厂中的工厂函数。如下图所示，client 通过调用 Factory 类来生成对应的 Person 类，从而达到创建对象的目的。



对于各种数据的操纵类也可以使用工厂方法来生成。如下图所示，client 通过 Person 类来生成对应的 manipulator，这种对应关系即为一种平行关系。

在这种情况下，工厂方法用于连接平行的类的层次。但是考虑到各个数据总是一直存在的，我们在软件运行过程中并不需要产生新的操纵类，因此我们通过工厂类来生成并不合算，所以这里不会采用此方法。



c. 原型（Prototype）

原型这一设计模式的意图在于用原型的实例来创建对象，通过拷贝原型来完成这个目的。

与原型相类似的 C++/Java 实现是拷贝构造函数，但二者有些许差别。拷贝构造函数只有 C++/Java 标准的一种实现方式，但是原型却完全可以定义不同的代码实现方式。原型是一种抽象的概念，拷贝构造函数可以被看成是它的一种实现方式。

当我们实例化多个几乎相同的 room 类时，我们可以用原型设计模式。使用这个模式，可以通过拷贝已有实例来进行实例化。在 Java 中可以使用拷贝构造函数来简单地实现这一设计模式。

d. 单件 (Singleton)

当我们需要保证一个类最多只有一个实例时，并且让它能够被全局访问时，我们可以使用单件模式。

与单件模式相类似的是语言中固有的静态类与全局变量 (C++)。相比于全局变量，单件模式避免了命名空间的污染。静态类的生命周期是从程序开始运行到程序终结，但是单件可以做到中途才实例化，到一定阶段也可以被析构，从而在生命周期上显得更灵活。

对于各个数据进行的操作被包裹在单独的类中，这个类可以应用单件模式。因此，对于前面所阐述的工厂类我们也可以应用单件模式，因为我们只需要一个工厂，而不需要多个。具体地，我们在 Factory 类中使用静态的 Instance() 函数来返回这个唯一的实例，并且将构造函数的可访问性为 protected，这可以防止意外的实例化所导致多个实例的情况。



我们项目的数据库操作类也可以使用单件模式。因为这些类只包含操作，没有必要有多个实例。而且，利用单件模式，可以避免数据库操作的冲突。而且，单件具有较好的扩展性，可以通过子类继承的方式对操作进行一些列拓展。

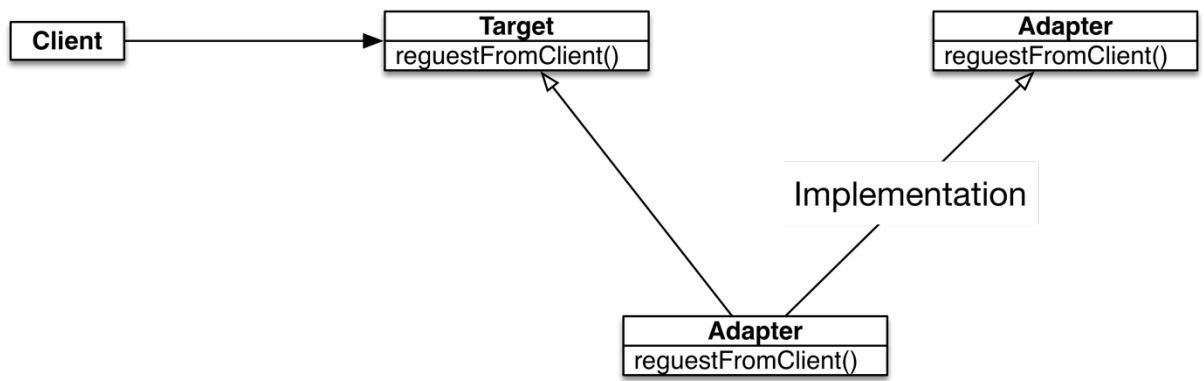


2. 结构型模式

a. 适配器（Adaptor）

适配器模式的重要作用在于使一些原本因为接口不兼容而无法一起工作的类能够在一起工作。

当代码有多人协作时，我们最初定义的类之间的接口调用可能不再得到实现，从而导致接口相互不兼容，因此我们可以利用适配器模式来实现接口的兼容。例如，我们可能已经开发了一个类 Adaptee，但是我们所开发的另一个却是以 Target 类的形式想要调用 Adaptee 这个类的，那么我们需要开发一个新的类 Adapter 去适配这一接口的不兼容情况。

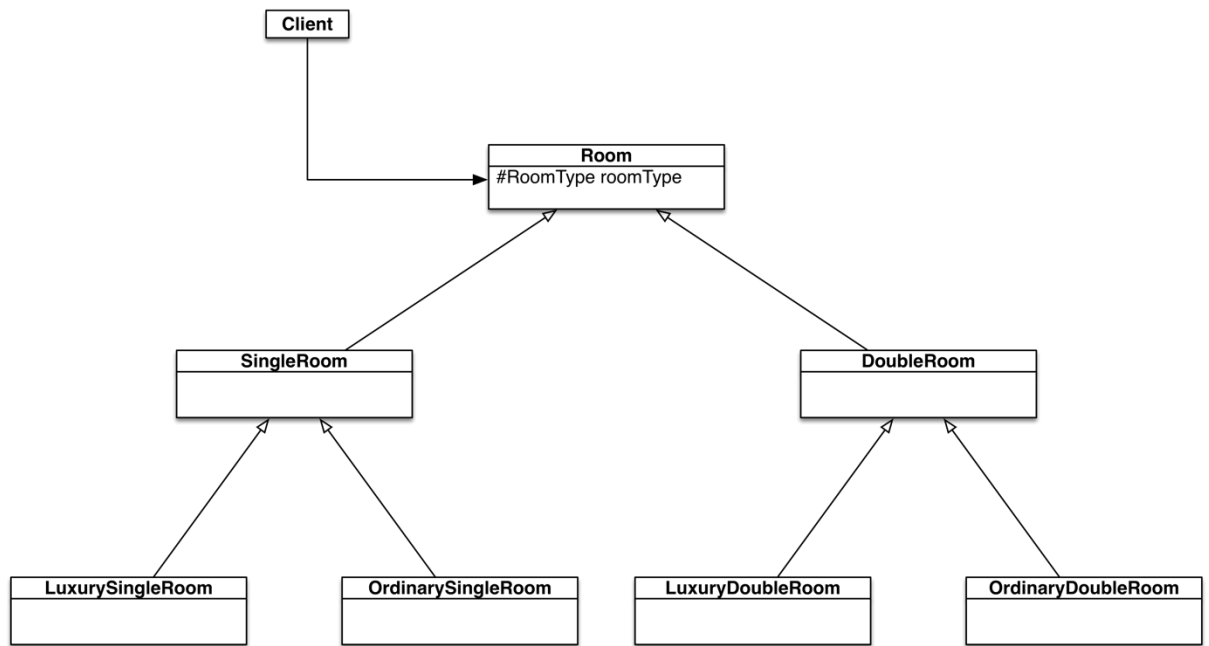


b. 桥接（Bridge）

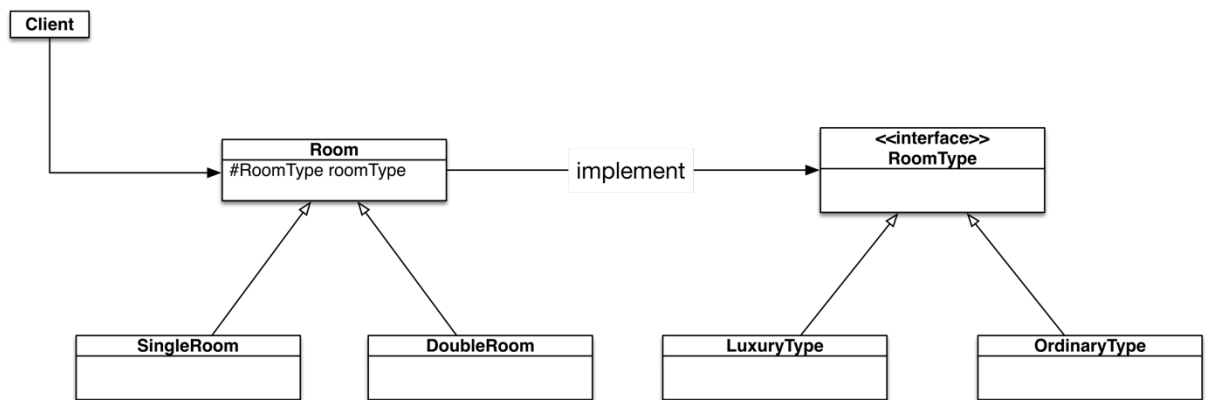
桥接常常用于当一个类有多个维度的属性时，不直接使用继承来进行复用。因为直接的继承会带来巨大的复杂度以及耦合度，桥接则是一种解耦合手段。具体地说，桥接常常用多继承（C++）/接口（Java）来实现。

在我们的系统中房间类既可以分为单人间、双人间，也可以分为豪华间和普通间。这种特性意味着我们可以用桥接模式去实现。

下图所示是未使用桥接模式的一种可能的实现方法，可以看到，由于豪华/普通，双人/单人两个维度相互交错，所以继承体系十分复杂。



下图所示为重构为桥接模式后的类图。我们把豪华/普通这一属性分离出来，单独使用一个接口去表示它。从图中可以发现，原先的耦合度被降低，继承体系的复杂度也得到降低。



c. 外观（Facade）

外观意在提供为子系统的一组接口提供一个高层的一致接口，这一接口有利于降低系统的复杂性。

外观模式与适配器模式在本项目中的作用是类似的，都可以被用来解决由于合作编程中的不协调带来的一系列问题。在合作编程中，由于初始时对类的功能不明晰，各个成员写出来的相似的类的接口可能不一致，这是可以引入一个 Facade 对象来为他调用者提供一致的 API，而忽略每个类的接口可能不一致这样的事实。

d. 代理（Proxy）

代理模式为其他对象提供了一种代理以控制对这个对象的访问。对一个对象进行访问控制的一个原因是为了只有在我们确实需要这个对象时才对它进行创建和初始化，另一个原因则是为原有对象扩展其他自定义的功能，Spring AOP 便处于这个理由使用了代理模式。

面向切面编程（AOP）是软件工程中的一种常见模式，它将分散在类、函数中的横切关注点抽象成了切面这一概念，并通过一些特殊的语法将其表达出来。AspectJ [7] 规定了一种特殊的 DSL 来表达切面，不过通常需要使用其提供的编译工具来处理这些特殊的语法；Spring AOP 则使用 XML 或 Java 注解来定义切面，这种实现使用到了代理模式。下面是一段基于注解的 Spring AOP 代码：

```
@Aspect
public class Audience {
    @Pointcut("execution(** concert.Performance.perform(..))")
    public void performance() {}
    @Before("performance()")
    public void silenceCellPhones() {
        System.out.println("Silencing cell phones");
    }
    @AfterReturning("performance()")
    public void applause() {
        System.out.println("CLAP CLAP CLAP!!!");
    }
}
```

根据我们的代码，手机静音方法将会被插入到表演之前，而鼓掌方法则将被插入至表演成功返回之后。为了实现这样的功能而又不去修改编译器或是虚拟机，一个行之有效的方法便是创建代理类。Spring 框架会为上述的 Performance 对象创建一个代理类，当其 perform 方法被调用时，代理类会先执行 AOP 的前置通知，在成功返回后又会调用其返回通知，这就实现了上述 AOP 注解的全部功能。这套技术也用到了 Spring IoC 容器的特性，在为 Bean 进行装配时，AOP 代理类对象而非实际对象会被注入进去，由于两者的方法签名完全一致，这个变动不会被使用者察觉到。

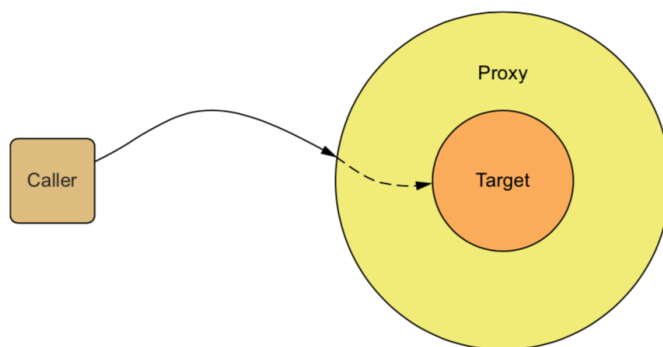


Figure 4.3 Spring aspects are implemented as proxies that wrap the target object. The proxy handles method calls, performs additional aspect logic, and then invokes the target method.

3. 行为模式

a. 解释器（Interpreter）

如果一种特定类型的问题发生的频率足够高，那么可能就值得将该问题的各个实例表述为一个简单语言中的句子。这样就可以构建一个解释器，该解释器通过解释这些句子来解决该问题。

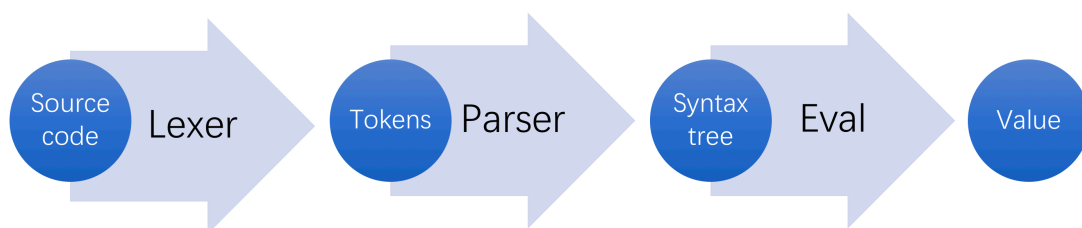
在我们的项目中，一个典型的例子是 Spring 框架[8]中的 Spring Expression Language（SpEL）。在 Spring 通过依赖注入技术为 Bean 进行装配时，即可使用 SpEL

为属性和构造器参数赋值，除此之外，Spring Security、Thymeleaf 等模块也可使用 SpEL 来进行表达。SpEL 在设计上是独立于 Java 语言的，Spring 自己实现了一个解释器对 SpEL 进行求值，这就是我们所说的解释器模式。

在装配 Bean 时，我们有两种方式使用 SpEL，一是在 XML 中书写 `<property name="hundred" value="#{100}">`，二是使用 Java 注解在属性定义前面写上 `@Value("#{100})`。当然我们也可以在 Java 代码中独立使用 SpEL，这时我们需要创建一个 `ExpressionParser` 的实例将 SpEL 字符串解析为 `Expression` 的实例，具体代码如下：

```
ExpressionParser parser = new SpelExpressionParser();
Expression exp = parser.parseExpression("'Hello World'");
String message = (String) exp.getValue();
```

对于一个解释器的实现，通常会有词法分析、语法分析、语义分析、执行求值等过程，词法分析负责对源代码进行分词生成符号序列，语法分析负责根据形式文法从符号序列生成抽象语法树，语义分析负责在现有语法树的基础上维护符号表、进行类型检查等等。其过程可以用下面的简图描述：



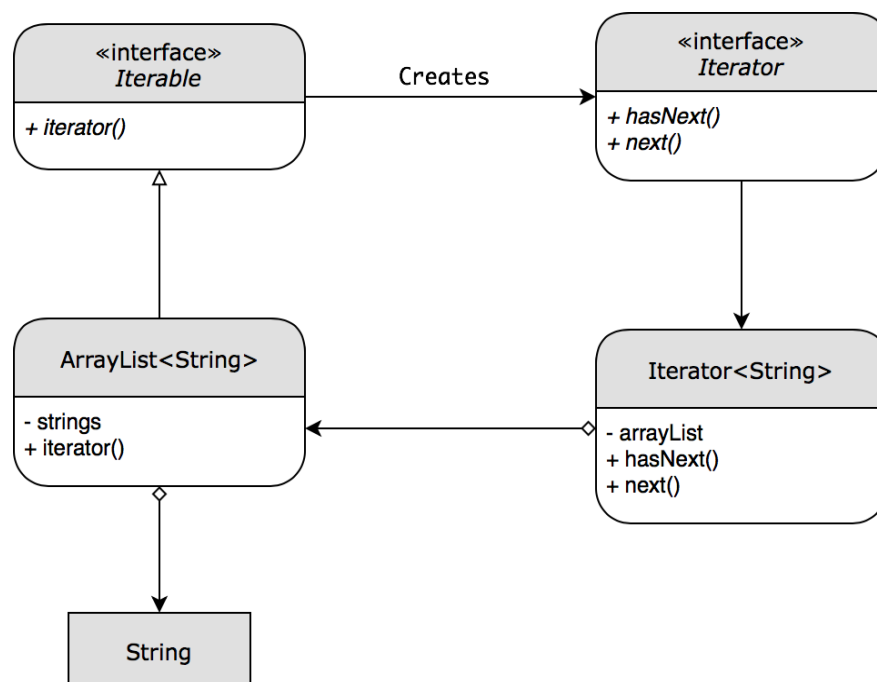
b. 迭代器 (Iterator)

迭代器提供了一种方法顺序访问一个聚合对象中各个元素，而又不需暴露该对象的内部表示。

在使用 Java 语言时，其实我们在不经意之间就用到了迭代器模式。Java 语言拥有 `foreach` 语句，也就是说我们不必像 C 语言那样枚举下标，即可直接使用形如 `for (String s : array) { System.out.println(s); }` 来遍历数组。而在 Java 内部的机制中，`foreach` 语句实际上就是迭代器的语法糖，它等价于如下代码：

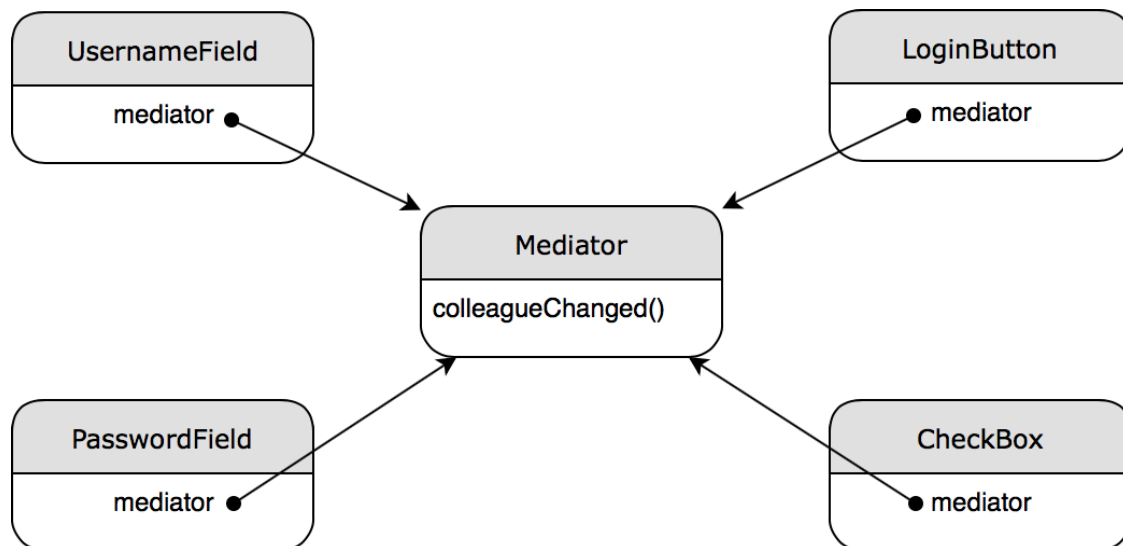
```
for (Iterator<String> i = array.iterator(); i.hasNext();) {  
    String s = i.next();  
    System.out.println(s);  
}
```

在这个例子中，`array` 变量所属的 `ArrayList` 类实现了 `Iterable` 接口，其 `iterator` 方法会返回一个迭代器的实例，而 `next` 方法则会每次返回数组的下一个元素。针对以上结构，我们可以画出如下类图：



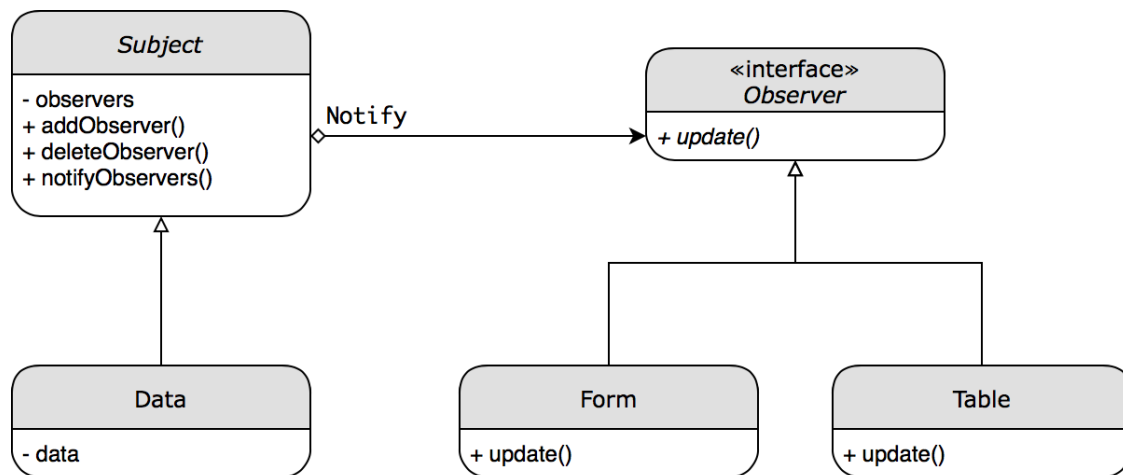
c. 中介者 (Mediator)

中介者模式用一个中介对象来封装一系列的对象交互。中介者使各对象不需要显式地相互应用，从而使其耦合松散，而且可以独立地改变它们之间的交互。一个对中介者模式的典型应用是登录页面的展示：在我们的项目中，用户分为用户登录和游客访问，用户登录需要用户名和密码，而游客访问不需要；另一方面，在用户未输入用户名时，我们将禁用密码框和登录按钮，在输入了用户名之后密码框将启动，在输入了密码之后登录按钮将启用。在这样一个复杂的设计中，如果将控制显示的逻辑分散在各个组件的类中，这不管对于编写代码还是调试代码都非常麻烦。这时一个良好的设计就是使用中介者模式，即不让各个对象之间互相通信，而是增加一个中介者来协调对象之间的关系。这样我们可以将控制显示的逻辑代码全部写在中介者类中，每当有界面更新就通知中介者来进行决断，大大降低了页面组件之间的沟通成本。



d. 观察者（Observer）

观察者定义了对象间的一种一对多的依赖关系，当一个对象的状态发生改变时，所以依赖于它的对象都得到通知并被自动更新。在我们的项目中，我们将这个模式应用在了数据修改通知界面更新这一机制上。在这里数据本身是观察者的目标，而用到这一数据的各个界面元素则是观察者。每当数据被修改时，它会通知所有它的观察者这一改动，而观察者们也跟根据这一通知来对界面元素进行更新，防止我们仍在展示过期的数据。这个模式的类图如下：

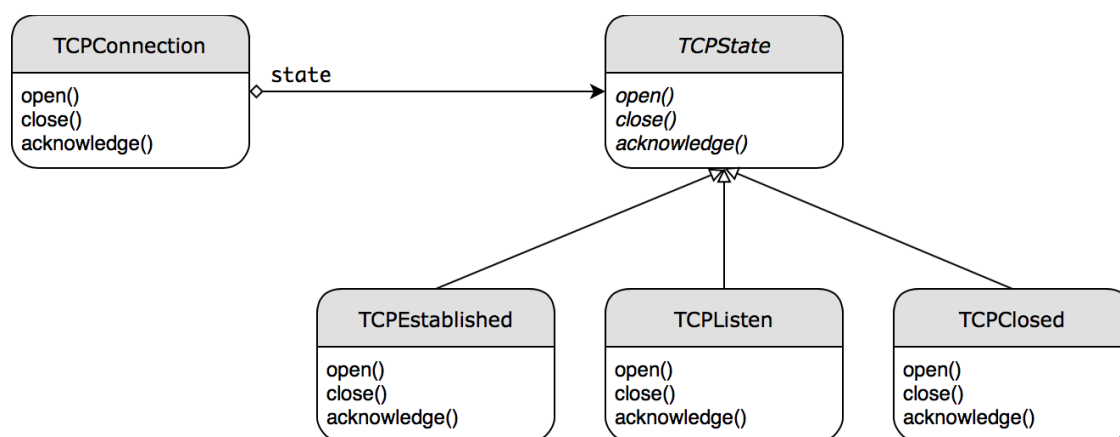


在这个模式中，目标和观察者是解耦合的，因为目标只知道有一系列实现了 **Observer** 接口的对象在观察它，而并不知道它们具体属于哪个类。同时，对观察者的通知操作是广播的形式，不需要指定任何特定对象，我们也可以随时对观察者列表进行增加和删减，这完全不会影响到观察者模式的正常实行。

e. 状态（State）

在面向对象程序设计中，类在不少情况下对应着真实世界的物体，但也有不存在于真实世界的情况，譬如状态。我们可以使用类来表示一种状态，通过切换类来变更

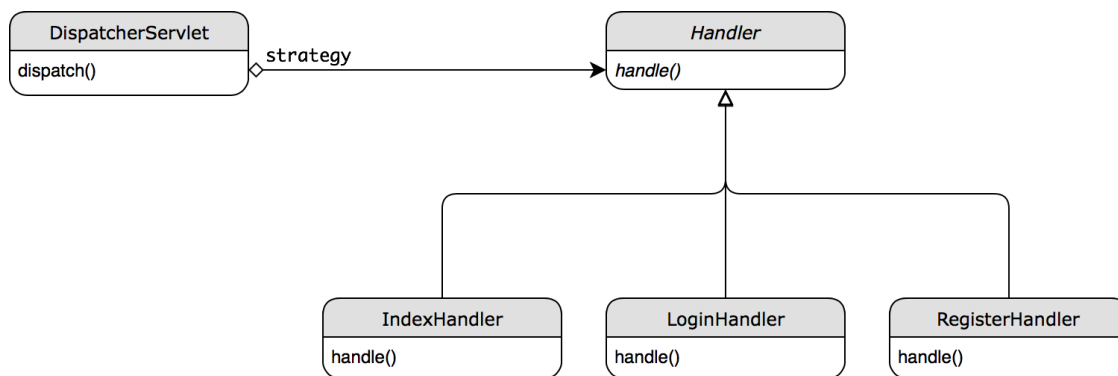
对象的状态，TCP 连接便是一个典型的应用。我们可以通过 TCPState 这个抽象类表示网络连接的状态，而其子类 TCPEstablished、TCPListen、TCPClosed 则表示各种具体的状态。TCPConnection 类维护着表示 TCP 当前连接状态的对象，并随着状态的改变切换状态对象的类，譬如当连接从已建立状态转为关闭状态时，则用 TCPClosed 实例取代原来的 TCPEstablished 实例。



f. 策略（Strategy）

在策略模式中，我们会定义一系列的算法，把它们一个个封装起来，并且使它们可相互替换。本模式使得算法可独立于使用它的客户而变化。

在我们的项目中，我们经常会用到策略模式来将不同的算法封装成相同的接口，这样它们就可以毫无费劲地互相替换了。其中一个典型的例子是 Spring MVC [8]中的控制器，对于各种不同的控制器实现，它们的函数签名都几乎完全相同，均为 `@RequestMapping("/path") public String handler(ModelMap map) {...}`。因为这里使用了 Java 注解，所以对象之间的关系不是非常明朗，我们可以将其改画成一种等价的形式：

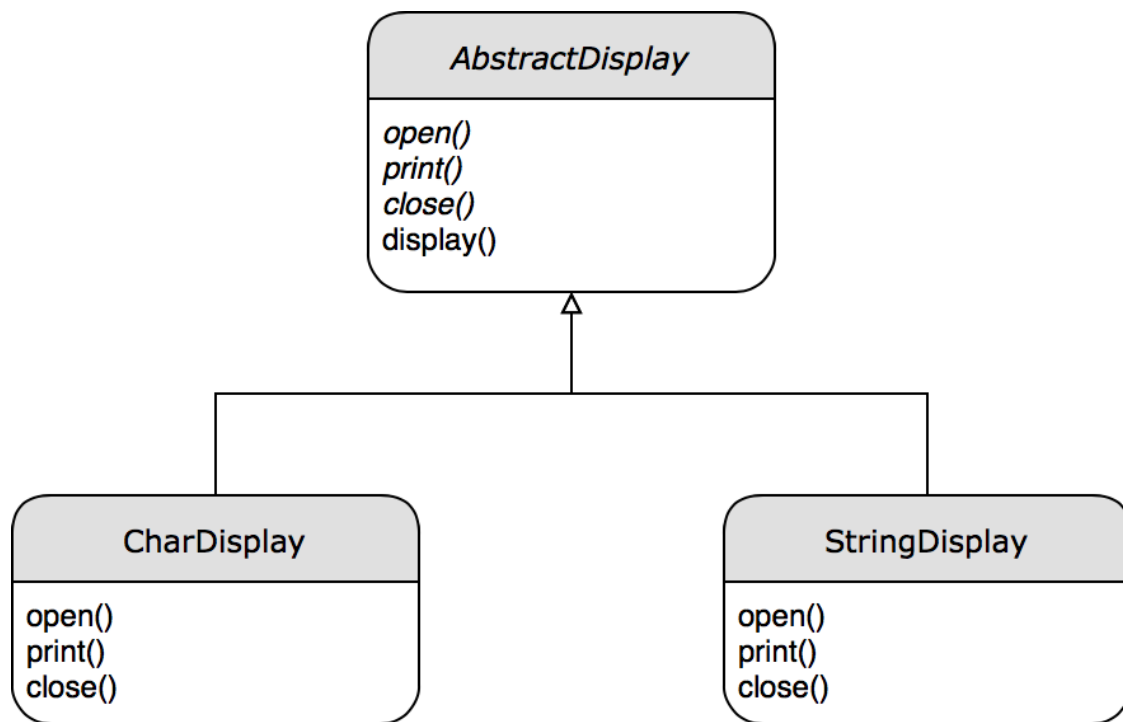


在该模式中，我们将处理不同 URL 的控制器实现为 Handler 的子类，而具体的处理代码则放在 handle 方法中。另一方面，DispatcherServlet 将在 dispatch 方法中调用控制器对象的 handle 方法，对于 DispatcherServlet 对象来说，不同的控制器都拥有相同的接口，因此我们可以在其不知情的情况下任意修改控制器的实现，这就极大地便利了我们的后端开发。

g. 模板方法（Template Method）

模板方法定义了一个操作中的算法的骨架，而将一些步骤延迟到子类中。本模式使得子类可以不改变一个算法的结构即可重定义该算法的某些特定步骤。

在我们的项目中，我们也经常使用模板方法模式来组织一些抽象方法的步骤，而这些抽象方法会在不同的子类中实现。例如我们在打印日志时在字符串前后加入一些标记，我们就可以先设计一个抽象的父类 AbstractDisplay，在其显示字符串的方法 display 中，可以用抽象方法 open、print、close 来组织显示的步骤，而不必在意这些方法具体是怎么写的。其类图如下：



IV. 参考文献

- [1]. Design pattern detection based on the graph theory. (2017). Design pattern detection based on the graph theory, 120, 211–225. <http://doi.org/10.1016/j.knosys.2017.01.007>
- [2]. Yu, D., Zhang, P., Yang, J., Chen, Z., Liu, C., & Chen, J. (2018). Efficiently detecting structural design pattern instances based on ordered sequences. *Journal of Systems and Software*, 142, 35–56. <http://doi.org/10.1016/j.jss.2018.04.015>
- [3]. Bou, C., Laosen, N., & Nantajeewarawat, E. (2018). Design Pattern Ranking Based on the Design Pattern Intent Ontology (Vol. 10751, pp. 25–35). Presented at the Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics), Cham: Springer International Publishing.
- [4]. Hunhs, M. N. and M. P. Singh (1997). "Ontologies for Agents". In: *IEEE-Internet Computing* 1, no. 6 (Nov.- Dec. 1997). pp. 81-83.
- [5]. A. De Nicola, M. Missikoff, R. Navigli (2009). "A Software Engineering Approach to Ontology Building". *Information Systems*, 34(2), Elsevier, 2009, pp. 258-275.
- [6]. Johnson, R., Gamma, E., Vlissides, J., & Helm, R. (1995). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

- [7]. Design pattern implementation in Java and AspectJ. (2002). Design pattern implementation in Java and AspectJ.
- [8]. Walls, C. (2014). Spring in Action. Manning.
- [9]. Nagel, Christian; Evjen, Bill; Glynn, Jay; Watson, Karli; Skinner, Morgan (2008). "Event-based Asynchronous Pattern". Professional C# 2008. Wiley. pp. 570–571. ISBN 0-470-19137-6.